TESTAREA SISTEMELOR SOFTWARE · PROIECT DE LABORATOR

Smart Parking Gate

Testare funcțională, structurală și prin mutații pentru motorul de decizie al unei parcări corporate multi-tenant

Natalia Moroz — 50%

Cerințele 1, 2, 5 · testare funcțională, acoperire, mutant echivalent

Victor Mura — 50%

Cerințele 3, 4, 6, 7 · CFG, MC/DC, mutanți ne-echivalenți, unelte

80
teste automate

V(G) = 7
complexitate ciclomatică

5
teste MC/DC (N + 1)

2
unelte de mutație

Universitatea din București · FMI · Master Ingineria Sistemelor Software, anul II · 2026

Echipa

7 cerințe, două perspective, o singură funcție

01
Testare funcțională
EP · BVA · Graf cauză-efect

02
Acoperire de cod
pytest-cov, fiecare set separat

03
Graf de control
CFG · V(G) · căi independente

04
MC/DC
decizia compusă EV, 5 teste

05
Mutant echivalent
ordin 1, demonstrație pe $\mathbb{Z}$

06
Mutanți ne-echivalenți
unul ucis, unul supraviețuitor

07
Unelte de mutație
mutmut vs. cosmic-ray

Natalia Moroz — 50%

Victor Mura — 50%

Culoarea fiecărui slide indică autorul secțiunii.

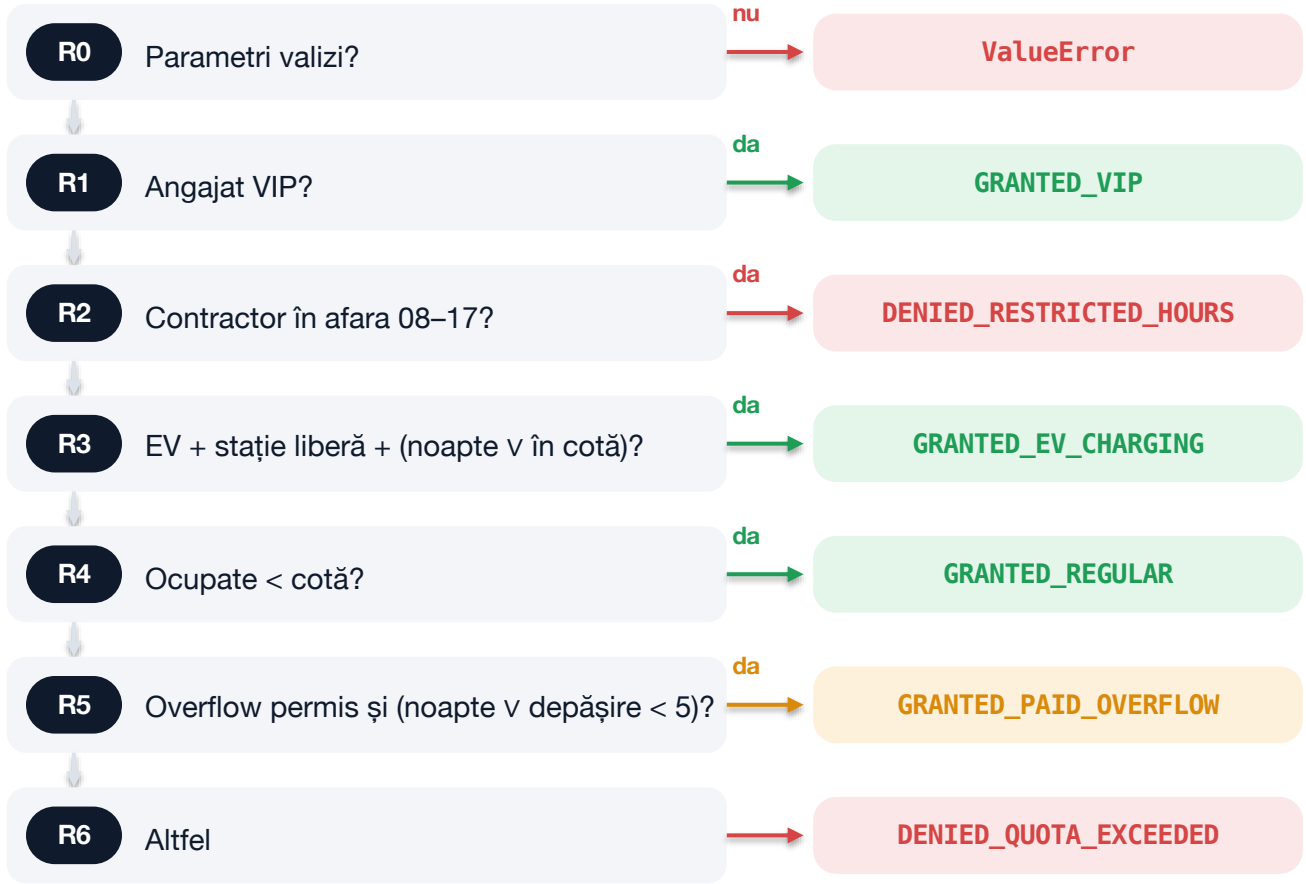
Echipa

Bariera unui campus de birouri cu mai mulți chiriași

SCENARIU

- **Mai multe companii** împart aceeași parcare; fiecare are o cotă contractuală de locuri gratuite.
- **O singură barieră** decide instant accesul pe baza a 7 parametri.
- **Resurse comune:** stații de încărcare EV și o zonă de overflow plătită de companie.
- **Funcție pură, deterministă** — fără I/O, ideală pentru testare unitară și mutații.

IERARHIA DECIZIILOR — evaluate_parking_access()



Funcția sub test și domeniile de intrare

```
def evaluate_parking_access(quota, occupied, hour, tier,
        is_ev, has_ev_spots, allows_overflow):
    if quota <= 0 or occupied < 0 or hour < 0 or hour > 23:
        raise ValueError(...)
    is_peak = (8 <= hour <= 17)
    if tier == VIP:
        return GRANTED_VIP
    if tier == CONTRACTOR and not is_peak:
        return DENIED_RESTRICTED_HOURS
    within = occupied < quota
    if is_ev and has_ev_spots and (not is_peak or within):
        return GRANTED_EV_CHARGING
    if within:
        return GRANTED_REGULAR
    overflow = occupied - quota
    if allows_overflow and (not is_peak or overflow < 5):
        return GRANTED_PAID_OVERFLOW
    return DENIED_QUOTA_EXCEEDED
```

Simplificat pentru lizibilitate — sursa completă: src/parking_gate.py. Linia evidențiată = decizia MC/DC.

Parametru	Tip	Domeniu valid
company_allocated_quota	int	≥ 1
company_current_occupied	int	≥ 0
entry_hour	int	0 … 23 (vârf: 8 … 17)
tier	Enum	REGULAR · VIP · CONTRACTOR
is_electric_vehicle	bool	True / False
has_ev_spots_available	bool	True / False
company_allows_overflow	bool	True / False

Domeniul de ieșiri (I1 … I6 + I7: ValueError)

I2 VIP | I3 EV | I1 REGULAR

I4 OVERFLOW | I6 HOURS ✕ | I5 QUOTA ✕

PARTEA I · NATALIA MOROZ

Testare funcțională

Cerința 1	Partiționare în clase de echivalență, valori de frontieră, graf cauză-efect
Cerința 2	Acoperirea de cod obținută de fiecare set, măsurată separat

Natalia Moroz

Partiționare în clase de echivalență (EP)

Parametru	Clase valide de intrare (simbol & domeniu)	Clase invalide (ValueError)
quota (q)	Q1: q ≥ 1	Q2: q = 0 · Q3: q < 0
occupied (o)	O1: o < q (sub cotă) · O2: o = q · O3: o > q	O4: o < 0
entry_hour (h)	H1: 0–7 (dimineață) · H2: 8–17 (vârf) · H3: 18–23	H4: h < 0 · H5: h ≥ 24
tier (t)	T1: VIP · T2: REGULAR · T3: CONTRACTOR	— (garantat Enum)
(ev, sp)	EV1: (T, T) · EV2: (T, F) · EV3: (F, –)	—
allows_overflow	OVF1: True · OVF2: False	—

DOMENIUL DE IEȘIRI (Curs: I1 … I7)

- I1: GRANTED_REGULAR
- I2: GRANTED_VIP
- I3: GRANTED_EV
- I4: PAID_OVERFLOW
- I5: DENIED_QUOTA
- I6: DENIED_HOURS
- I7: ValueError

13

cazuri de test (EP)

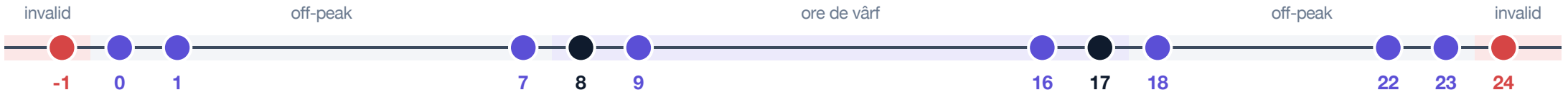
6 teste pe invalide (Q2..H5 ⇒ I7)
7 teste pe valide (acoperă I1..I6)
tests/test_ep.py

Regula cursului (Myers): Câte un reprezentant per clasă; clasele invalide se testează izolat („one at a time”) pentru a preveni mascarea defectelor.

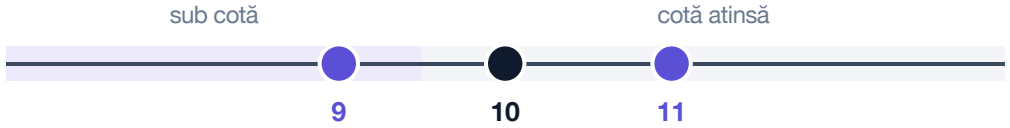
Natalia Moroz

Analiza valorilor de frontieră (BVA)

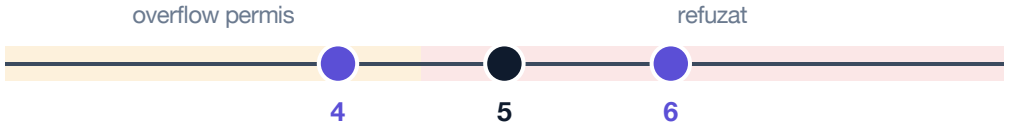
entry_hour — limitele zilei și ale intervalului de vârf



occupied față de quota = 10



depășire la ore de vârf (occupied – quota)



Condiție / Variabilă	Formulă Curs (Frontiere)	Valori de test	Eroare prevenită
quota ≥ 1	Min−1, Min, Min+1	`0 · 1 · 2`	cote nule sau negative (ValueError)
occupied ≥ 0	Min−1, Min, Min+1	`−1 · 0 · 1`	contoare negative (ValueError)
occupied < quota	q−1, q, q+1 (q=10)	`9 · 10 · 11`	< confundat cu ≤ la limită
overflow < 5	Max−1, Max, Max+1	`4 · 5 · 6`	depășire strictă a pragului

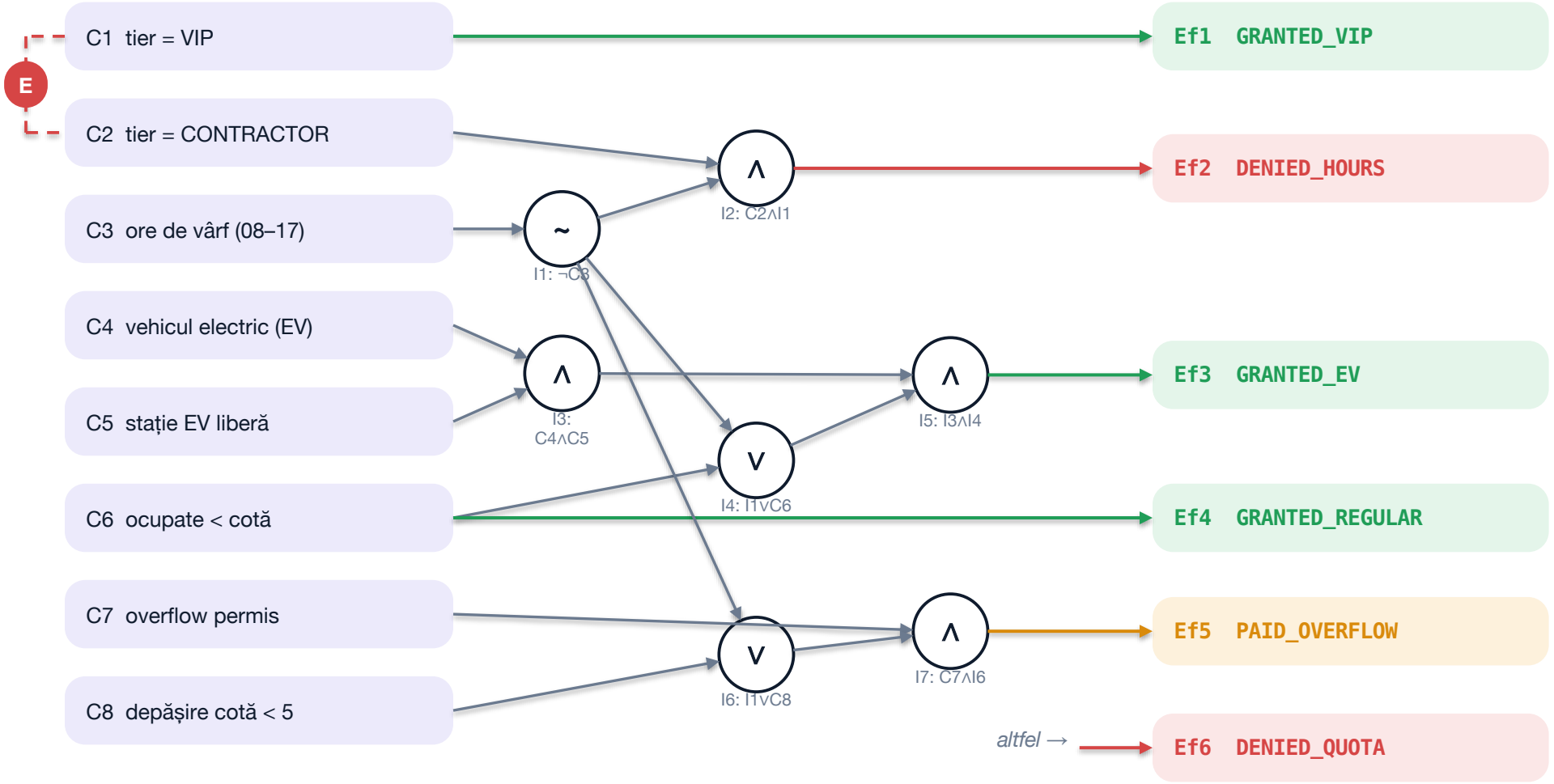
24

cazuri de test (BVA)

Metoda cursului (Myers): pe interval [a, b] se testează {a−1, a, a+1} și {b−1, b, b+1}.
Detectează erori off-by-one și inegalități nestricte.
tests/test_bva.py

Natalia Moroz

Graful cauză-efect (CEG) — Sintaxă Myers



Sintaxă Curs (Myers)

Relații logice:

—> Identitate (if C)
~ NOT (if ¬C)
∧ AND (if C1 ∧ C2)
∨ OR (if C1 ∨ C2)

Constrângere cauze:

E(C1, C2) Exclusivitate
C1 + C2 ≤ 1 (tier unic)

Constrângeri efecte:

M(Efi, Efj) Mascare
Ef1 > Ef2 > Ef3 > Ef4 > Ef5 > Ef6
Prioritate decizională cascadă

Natalia Moroz

Tabelul de decizie Myers: 9 reguli de testare

Condiție / Efect	R1	R2	R3	R4	R5	R6	R7	R8	R9
C1 tier = VIP	1	0	0	0	0	0	0	0	0
C2 tier = CONTRACTOR	0	1	0	0	0	0	0	0	0
C3 ore de vârf (08–17)	–	0	0	1	–	0	1	1	–
C4 vehicul electric	–	–	1	1	0	0	0	0	0
C5 stație EV liberă	–	–	1	1	–	–	–	–	–
C6 ocupate < cotă	–	–	–	1	1	0	0	0	0
C7 overflow permis	–	–	–	–	–	1	1	1	0
C8 depășire cotă < 5	–	–	–	–	–	–	1	0	–
Ef1 GRANTED_VIP	1	0	0	0	0	0	0	0	0
Ef2 DENIED_HOURS	0	1	0	0	0	0	0	0	0
Ef3 GRANTED_EV	0	0	1	1	0	0	0	0	0
Ef4 GRANTED_REGULAR	0	0	0	0	1	0	0	0	0
Ef5 PAID_OVERFLOW	0	0	0	0	0	1	1	0	0
Ef6 DENIED_QUOTA	0	0	0	0	0	0	0	1	1
Decizie așteptată	Ef1	Ef2	Ef3	Ef3	Ef4	Ef5	Ef5	Ef6	Ef6

Format standard de curs (Myers): N = p + q = 14 linii de cauze/efecte, 9 reguli pe coloane · tests/test_ceg.py

Procedura de Derivare (Conform Cursului)

1. Inițializare: nr_coloane = 0 (tabel inițial gol)

2. Backtracking per efect (Efi = 1):

- **Ef1 = 1** ⇒ C1=1 ⇒ **Regula 1 (R1)** (VIP)
- **Ef2 = 1** ⇒ C2=1 ∧ ¬C3=1 ⇒ **Regula 2 (R2)** (ore)
- **Ef3 = 1** ⇒ C4 ∧ C5 ∧ (¬C3 ∨ C6) ⇒ **R3, R4** (EV)
- **Ef4 = 1** ⇒ C6=1 ⇒ **Regula 5 (R5)** (regular)
- **Ef5 = 1** ⇒ C7 ∧ (¬C3 ∨ C8) ⇒ **R6, R7** (overflow)
- **Ef6 = 1** ⇒ ¬C6 ∧ … ⇒ **R8, R9** (refuz cotă)

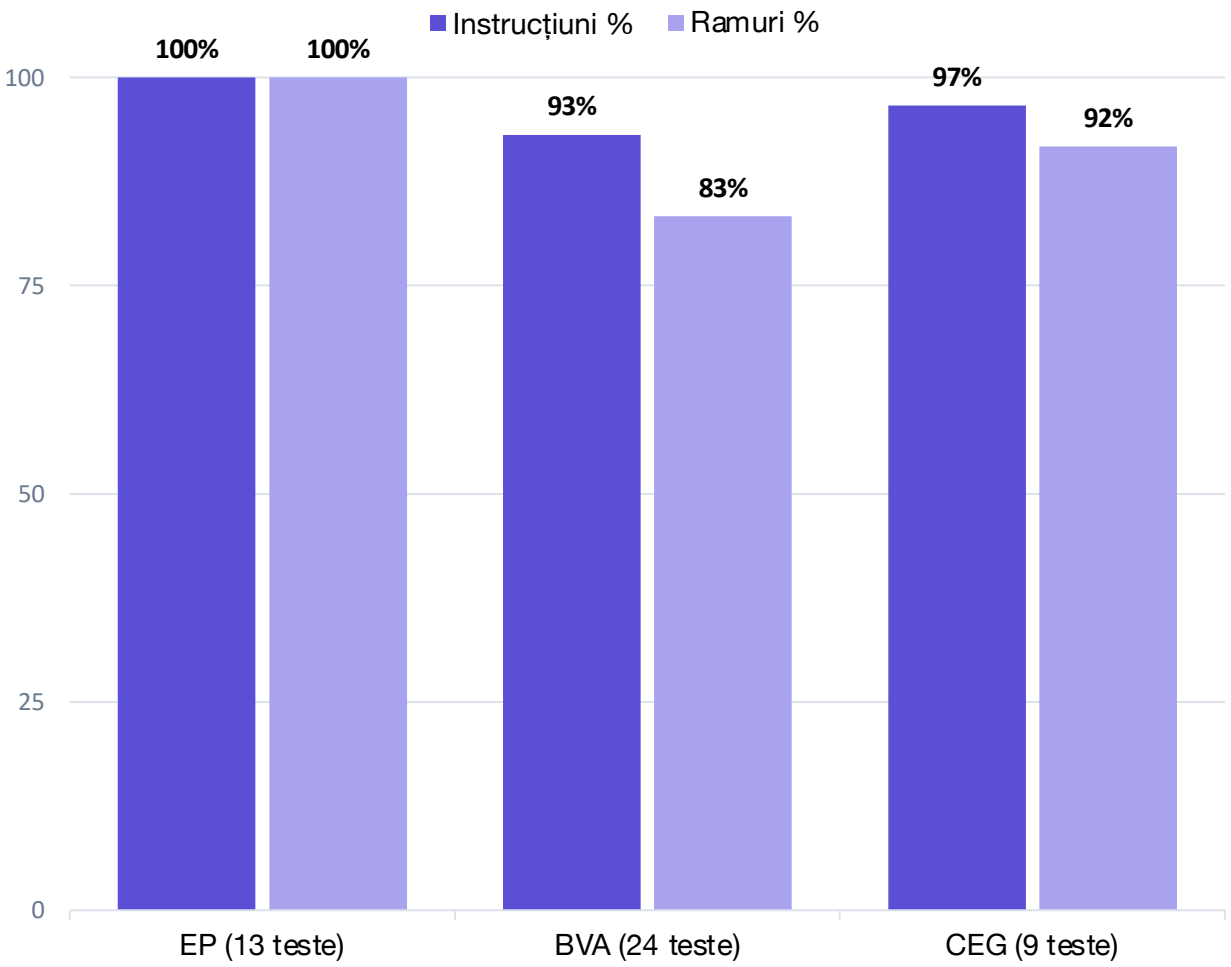
3. Constrângeri & Măști (Curs):

- E(C1, C2) Exclusivitate: C1 + C2 ≤ 1 (tier unic)
- M(Efi, Efj) Măști: Ef1 > Ef2 > Ef3 > Ef4 > Ef5 > Ef6 elimină combinațiile redundante sau mascate decizional.

4. Tabel N = p + q = 14 linii, M = 9 reguli: 9 teste automate minimale în tests/test_ceg.py.

Natalia Moroz

Același cod, trei seturi de teste, trei rezultate



EP (13 teste) — **100%**

`29/29 instrucțiuni, 12/12 ramuri`

O clasă per rezultat (I1..I6) + clasele invalide (I7) ⇒ fiecare ramură e atinsă.

BVA (24 teste) — **93% / 83%**

`lipsesc liniile 61 și 70`

BVA vizează granițele numerice; deciziile calitative VIP și EV nu au frontiere comensurabile.

CEG (9 teste) — **97% / 92%**

`lipsește linia 54 (ValueError)`

Graful modelează reguli de business valide; excepțiile de intrare sunt în afara specificului CEG.

pytest --cov=src --cov-branch, rulat separat pe fiecare set · src/parking_gate.py

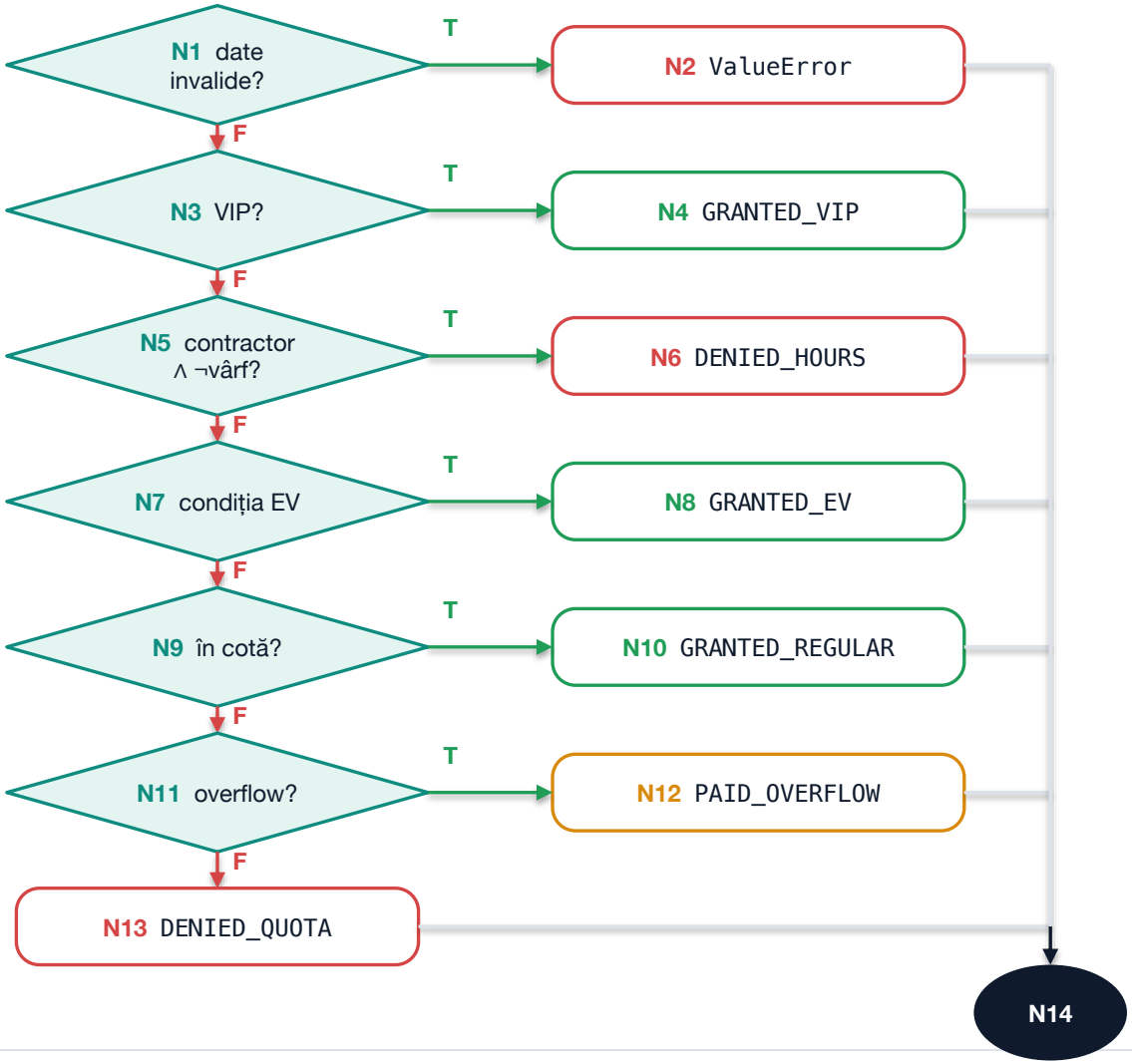
PARTEA II · VICTOR MURA

Testare structurală

Cerința 3	Graful de control, complexitatea ciclomatică, căile independente
Cerința 4	Modified Condition / Decision Coverage pe decizia compusă EV

Victor Mura

Graful de flux de control



COMPLEXITATE CICLOMATICĂ (McCabe)

V(G) = 7

n = 14 noduri
e = 19 arce
P = 6 predicate

Arce și noduri (Curs)

`V(G) = e − n + 2p`

`19 − 14 + 2(1) = 7  (sau e − n + 1 = 7)`

Noduri predicat

`V(G) = P + 1`

`6 decizii binare + 1 = 7`

Regiuni plane

`V(G) = R`

`6 regiuni închise + 1 exterioară = 7`

Trei formule de curs independente confirmă V(G) = 7 căi în setul de bază.

Victor Mura

7 căi liniar independente, fiecare cu testul ei

Cale	Secvență de noduri	Date de test	Rezultat
P1	N1 → N2	quota = 0	ValueError
P2	N1 → N3 → N4	tier = VIP	GRANTED_VIP
P3	N1 → N3 → N5 → N6	CONTRACTOR, ora 21	DENIED_RESTRICTED_HOURS
P4	… → N5 → N7 → N8	EV, stație liberă, ora 6	GRANTED_EV_CHARGING
P5	… → N7 → N9 → N10	occupied 5 / quota 10	GRANTED_REGULAR
P6	… → N9 → N11 → N12	12 / 10, overflow, ora 10	GRANTED_PAID_OVERFLOW
P7	… → N9 → N11 → N13	10 / 10, fără overflow	DENIED_QUOTA_EXCEEDED

Setul de bază de căi independente (McCabe / Curs): Cele 7 căi formează o bază liniară în spațiul căilor de execuție. Fiecare cale adaugă cel puțin un arc nou netraversat anterior, asigurând 100% Branch Coverage (19/19 arce). Orice alt traseu prin funcție se exprimă ca o combinație liniară a acestor 7 căi fundamentale.

Victor Mura

MC/DC pe decizia de acces la stația EV

$$D = (A \wedge B) \wedge (\neg C \vee D)$$

A vehicul electric B stație liberă
C ore de vârf D ocupate < cotă

Test	A	B	C	D	Dec.	Justificare independență
T1	1	1	0	0	1	Bază comună (acces EV aprobat)
T2	0	1	0	0	0	A este 0 ⇒ acces refuzat
T3	1	0	0	0	0	B este 0 ⇒ fără stație liberă
T4	1	1	1	0	0	C este 1 ⇒ vârf, ¬C este 0
T5	1	1	1	1	1	D este 1 ⇒ sub cotă, resuscitează OR

Criteriul MC/DC (Standard DO-178C / Curs)

1. Fiecare condiție atomică (A, B, C, D) ia valorile True și False.
2. Decizia compusă D ia atât valoarea True cât și False.
3. Fiecare condiție influențează decizia independent (schimbând doar acea condiție și ținându-le fixe pe restul, decizia se schimbă).
Dimensiune: N + 1 = 5 teste minimale (reducere de 69% față de $2^4 = 16$).

PERECHI DE INDEPENDENȚĂ (Efect demonstrat conform cursului)

A **T1** 1 1 0 0 → 1 → **T2** 0 1 0 0 → 0

B **T1** 1 1 0 0 → 1 → **T3** 1 0 0 0 → 0

C **T1** 1 1 0 0 → 1 → **T4** 1 1 1 0 → 0

D **T5** 1 1 1 1 → 1 → **T4** 1 1 1 0 → 0

Set minimal: N + 1 = 5 teste · tests/test_mcdc.py · 5/5 teste trec

PARTEA III · ECHIPA

Testare prin mutații

- **Cerința 5** — Un mutant de ordin 1 echivalent — și de ce nu poate fi ucis
- **Cerința 6** — Doi mutanți ne-echivalenți: unul ucis, unul supraviețuitor
- **Cerința 7** — Generare automată cu mutmut și cosmic-ray

Natalia Moroz

Mutant de ordin 1 echivalent

```
# src/parking_gate.py, linia 57
- is_peak_hours = (8 <= entry_hour <= 17)
+ is_peak_hours = (7 < entry_hour < 18)
```

DEMONSTRAȚIE

1. entry_hour este de tip int, deci $h \in \mathbb{Z}$.

2. Pe $\mathbb{Z}$: $h \geq 8 \Leftrightarrow h > 7$ și $h \leq 17 \Leftrightarrow h < 18$.

3. Deci is_peak_hours are aceeași valoare pentru orice h, iar restul codului e identic.

∴ **M(x) = P(x) pentru orice intrare x — niciun test nu poate ucide mutantul.**

VERIFICARE EXHAUSTIVĂ

1 152

comparații original vs. mutant, toate identice
24 ore × 3 roluri × 2^3 combinații bool × 2 niveluri de ocupare

Operator & Impact Teoretic (Curs)

- **Operator ROR (Relational Operator Replacement):**
  Înlocuirea intervalului închis cu interval deschis pe mulțimea $\mathbb{Z}$.

- **Impact asupra Scorului de Mutație:**
  Mutantul echivalent M(x) = P(x) nu poate fi ucis de niciun test.
  Conform cursului, se exclude din numitor: MS = D / (M – E).
  Păstrarea lui în numitor ar penaliza artificial calitatea testelor.

Victor Mura

Același test, doi mutanți, două destine

TESTUL ALES (din BVA)

`test_bva_occupied_exactly_at_quota`

`quota = 10 · occupied = 10 · ora 12 · REGULAR fără EV · allows_overflow = False`

Mutant #1 — UCIS

ROR: < devine ≤

```
- within = occupied < quota
+ within = occupied <= quota
```

- 10 <= 10 este True (stare internă alterată)
- funcția returnează GRANTED_REGULAR
- testul așteaptă DENIED_QUOTA_EXCEEDED
- **Strong Mutation: ieșire eronată ⇒ ucis (D)**

Mutant #2 — SUPRAVIEȚUIEȘTE

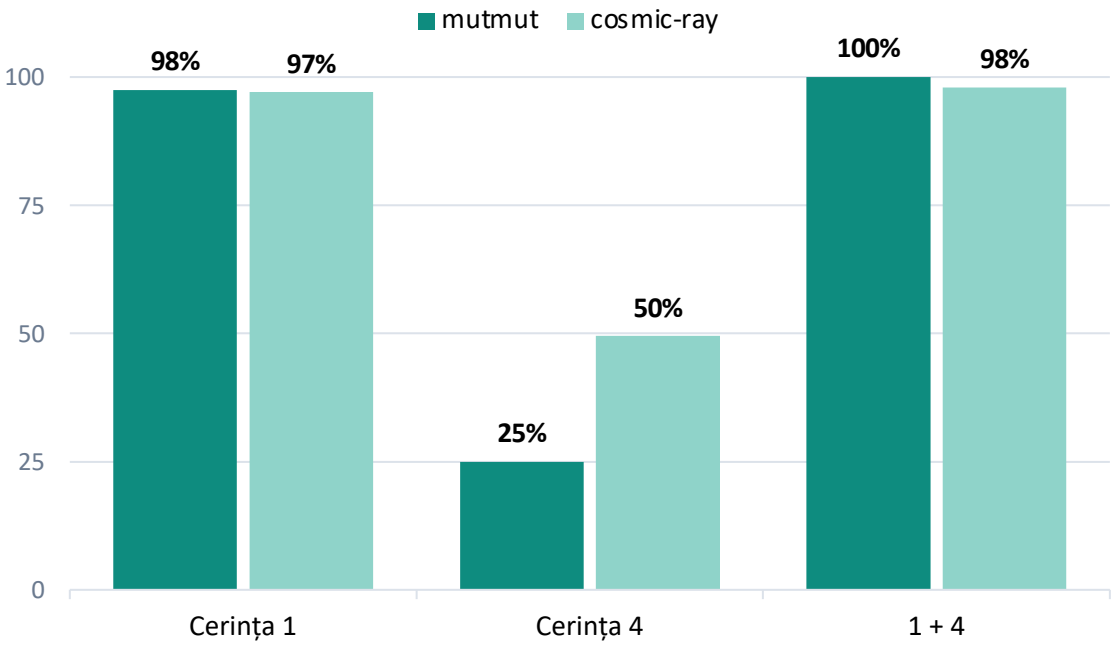
CRP: 5 devine 4

```
- ... (not is_peak or overflow < 5)
+ ... (not is_peak or overflow < 4)
```

- allows_overflow = False
- expresia `False and …` se scurtcircuitează
- pragul modificat nu e evaluat niciodată
- **Nici măcar weak mutation ⇒ supraviețuiește (L)**

Victor Mura

mutmut vs. cosmic-ray pe trei seturi de teste



Set de teste	mutmut ucis / total	cosmic-ray ucis / total
Cerința 1	39 / 40	98 / 101
Cerința 4	10 / 40	50 / 101
1 + 4	40 / 40	99 / 101

FORMULA SCORULUI DE MUTAȚIE (Curs)

$$MS(T) = D / (M - E) \times 100\% = D / (L + D) \times 100\%$$

D = uciși (distinși) · L = supraviețuitori (live) · E = echivalenți (excluși)

Operatori: mutmut vs cosmic-ray

mutmut generează 40 de mutanți (înlocuiri de literali, enum-uri și operatori relaționali). cosmic-ray generează 101 mutanți axați pe rotația completă a operatorilor de comparație (Lt_Eq, LtE_Lt), operatorilor booleeni (And_Or) și mutarea numerelor.

Analiză Supraviețuitori (Live Mutants)

La mutmut, setul combinat ucide 40/40 mutanți (100% scor de mutație!). La cosmic-ray, doar 2 mutanți din 101 supraviețuiesc (98.02% scor brut), ambii fiind mutanți matematic echivalenți (== înlocuit cu is pe membri Enum). Scorul de mutație ajustat este 100%.

Cerința 1 vs Combinat (1+4)

Setul funcțional (Cerința 1) oferă o acoperire largă a tuturor ramurilor (97.5% mutmut / 97.03% cosmic-ray). Setul MC/DC (Cerința 4) este hiperspecializat pe decizia compusă EV (ucide 100% din mutanții acesteia). Combinate, cele două seturi ating 100% eficiență de mutație.

Echipa

Ce am obținut și ce am învățat

80
teste, toate trec

100%
ramuri acoperite (EP)

7
căi independente

100%
scor mutmut, set combinat

Tehnicile funcționale se completează

BVA prinde erorile de frontieră, CEG combinațiile logice, iar EP acoperă clasele I1..I7.

Acoperirea structurală nu garantează robustețea

100% Branch Coverage nu detectează mutațiile subtile (ROR/CRP) — analiza de mutație arată calitatea reală.

Scurtcircuitarea ascunde defecte

Evaluarea leneșă din Python face ca mutanții din ramurile needecise să supraviețuiască fără MC/DC.

Utilizare AI

Asistentul Antigravity a fost folosit pentru schelet de cod, teste și documentație.

Prompturile, rezultatele și verificările sunt documentate în AI_USAGE_LAB.md, conform cerinței.

Cod, teste, documentație: `github.com/victormura/tss` · **Întrebări?**